

tradeforme — Next Steps

Action list of things only **you** can do. Everything below is blocked on your input, your credentials, or your decision — not on more code.

Where the project stands

Repository	https://github.com/EtixP/tradeforme (private)
Branch	main
Milestones done	M1, M2, M3 (deterministic extraction, 96.6% OK), M4, M5 (signals stored, v2 with KOSP)
Tests passing	108 / 108
Disclosures in DB	1,119,270 across 2021-12-29 to 2026-06-24 (4.5 years, Loop-12 extension)
Supply-contract events	3,531 candidates → 3,412 OK + 119 manual-review (96.6% / 3.4%), 0 blocked
Real contract values extracted	3,412 events with verified contract_value, prior_year_revenue, ratio
Candidate signals stored	693 (v2 strategy: ratio ≥ 0.08, KOSPI only, skip government counterparty). Mean strength
Price data source	pykrx (free, no auth)
LLM extraction	Code complete (prompts + validator + client) — optional now that deterministic parser wo
Broker integration	PaperBroker (historical fills) ready; KIS broker needs your credentials

Threshold sweep on 24 months / 3,412 events

Net of 0.313% roundtrip cost (commission + VAT + sale tax + slippage). Loop-3 ran the same sweep on only 514 events (3 months); the comparison column shows how those earlier numbers held up after 6.6× more data.

Threshold	n (24mo)	T+1 net	T+5 net	T+5 win%	PF (5d)	Median 5d	Loop-3 T+5 (n=514)
0.00 (all)	3,412	−0.12%	+0.35%	44.1%	1.21	−0.68%	+1.21% (n=514)
0.08 «	2,176	−0.07%	+0.22%	42.9%	1.16	−0.73%	+0.55% (n=333)
0.15	1,177	−0.16%	+0.56%	44.0%	1.25	−0.71%	+0.89% (n=169)
0.20	819	−0.06%	+0.59%	42.9%	1.25	−1.07%	+1.15% (n=110)
0.30	470	+0.18%	+0.50%	41.2%	1.21	−1.29%	+3.10% (n=59) «
0.50	217	+0.47%	+0.92%	38.1%	1.31	−1.79%	+2.67% (n=23)
1.00	66	−0.06%	−1.83%	32.3%	0.69	−3.98%	—

The big finding: the ≥ 0.30 "edge" from Loop 3 was largely noise. Going from 59 to 470 events, the T+5 net mean dropped from +3.10% to +0.50% and profit factor from 1.93 to 1.21. Median is negative at every threshold. The 1.00+ row is actually negative — suggests very large contracts may be a slight negative

signal (possibly representing distressed wins or unusual situations).

Honest conclusion: after 24 months and 3,412 events, the major-supply-contract event does NOT produce a clean tradable post-disclosure drift after costs. Mean net returns at every threshold are within noise of zero, win rates are 38–46%, medians are negative. This is exactly the kind of "strong negative result" CLAUDE.md anticipates — it answers the original research question. The market reacts efficiently to these disclosures by event-day close.

What can still rescue this: (a) sub-segment by counterparty type, sector, market cap, or recent volatility — maybe certain slices DO work. (b) try other event types (buybacks, dilutive financing, halt/resumption). (c) different exit horizons (T+10, T+20). (d) the M5b ML filter on richer features. Each is a real hypothesis to test — none is guaranteed to work.

Loop-6 subgroup + walk-forward results

Sliced the 3,377 events by market, ratio bucket, contract value, day-of-week, calendar quarter, and counterparty type (regex heuristic). Then walk-forward validated the most-positive subgroup and the most-negative subgroup across four 6-month windows. Honest reading below.

Subgroup	Aggregate T+5	Aggregate PF	Walk-fwd verdict (4 windows)
KOSPI / ratio 0.15-0.30	+1.59% (n=199)	1.59	ALTERNATES (+/-/-/+) — not robust
KOSPI (any ratio)	+0.63% (n=1708)	1.22	3/4 windows positive — modest but holds
KOSDAQ (any ratio)	+0.06% (n=1669)	1.02	no edge
Government counterparty	−2.14% (n=90)	0.51	3/4 windows NEGATIVE (sometimes −5%)
Other_korean_corp	−0.65% (n=417)	0.84	consistently negative
Ratio 1.00+	−1.83% (n=66)	0.64	small n; likely distressed/unusual events

Two actionable findings:

- **Government-counterparty contracts consistently underperform.** 3/4 walk-forward windows show net T+5 returns of –5% to –0.5%. Aligns with intuition: government contracts are pre-priced, low-margin, expected. We can't short (CLAUDE.md rule), but this is a **strong negative filter** — the strategy should skip long signals where counterparty is government.
- **KOSPI > KOSDAQ** in 3/4 windows. Aligns with the liquidity-premium hypothesis. Worth restricting the strategy to KOSPI-only.

Two findings ruled out: the KOSPI/0.15-0.30 subgroup that looked best in aggregate (+1.59% T+5) alternates positive/negative across windows — a classic small-sample fluke. The 0.30+ ratio "edge" from Loop 3 stays dead at scale. Median return remains negative at every threshold.

Raw CSV: data/subgroup_analysis.csv (3,377 events x 6+ feature columns).

Raw CSV: data/event_study_results.csv (3,531 events, all horizons, gross). Loop-3 log of the smaller dataset is preserved in git history.

Action items

Sorted by priority. P1 unblocks the next milestone. P2 hardens the existing system. P3 is optional polish.

Priority	What	Why	How
P1	Decide on the strategic pivot — Loop 100 needed as a plausible event	Loop 100 needed as a plausible event	Reply with advice path (verification) (2k words) (hard gaze) (30)
P1	Get an LLM API key (Anthropic / OpenAI) LLM needed for ML initiative	LLM needed for ML initiative	Go to OpenAI / Anthropic early type of language is strong. Address it then
P2	Rotate the DART API key	The current key briefly appeared in chat outputs during the ■■■■ Smoke Keltay (■■■■ the Rep late 06)	Change it during the ■■■■ Smoke Keltay (■■■■ the Rep late 06)
P2	Set up GitHub branch protection	Once live trading code exists, you don't want to be able to push to main	Go to GitHub.com → Settings → Branches → add protection
P2	Back up data/kdtb.db	111k rows of ingested DART data	Easy to replicate into a USB drive or back up to the Desktop or data
P2	Schedule daily DART ingestion	Right now ingestion only runs when you are home	Go to the CLI manually / Desktop or use cron to schedule it
P2	Obtain KIS broker credentials	Required for Milestone 6 (broker integration)	Go to the KIS website to get the credentials
P2	Decide whether to raise min_confidence threshold	Data from the loop's threshold sweep	Go to the config file and change the threshold
P2	Review risk thresholds in config	Default is 0.5	Go to the config file and change the threshold
P3	Install gitleaks pre-commit hook	Catches an accidental commit of .env	Go to the gitleaks website and install the pre-commit hook
P3	Decide if PDF action lists should be included	Currently in the STEPS.pdf will be included	Go to the PDF file and decide if it should be included
P3	Decide on Korean market holidays	Right now the ingestion CLI runs every day	Go to the Korean market website and get the holidays

Where to find things

Project root	/Users/jsp2022310/Desktop/tradeforme
Repo on GitHub	https://github.com/EtixP/tradeforme
Project spec	CLAUDE.md (in repo root)
Source code	src/kdtb/ — schemas, config, data, strategy, risk, storage
Tests	tests/ — run with: .venv/bin/pytest
DART ingestion CLI	scripts/ingest_disclosures.py --date YYYY-MM-DD
Event study CLI	scripts/run_event_study.py [--limit N]
Parse docs CLI	scripts/parse_supply_contracts.py [--limit N] [--skip-existing]
Filtered event study	scripts/run_filtered_event_study.py [--min-ratio 0.08]
Strategy runner	scripts/run_strategy.py [--min-ratio 0.30] [--dry-run]
DB backup	./scripts/backup_db.sh — gzipped SQL dump under data/backups/
Walk-forward	scripts/walk_forward.py — reproduces the v0/v1/v2 4-window table
Paper backtest	scripts/run_paper_backtest.py — realized-PnL with 3 execution assumptions
Per-category analysis	scripts/analyze_event_category.py --category <X>;
Cross-category summary	scripts/summarize_all_categories.py
Event study by category	scripts/run_event_study.py --category <X>;
Event blacklist (NEW)	src/kdtb/risk/event_blacklist.py — risk-engine check for recent negative events
Local DB (gitignored)	data/kdtb.db — SQLite, inspect with sqlite3 or DB Browser
Event study CSV	data/event_study_results.csv
Secrets (gitignored)	.env — DART_API_KEY lives here
Config	config/default.yaml, config/paper.yaml, config/live_tiny.yaml
Live-trading guard	src/kdtb/config.py — raises if YAML says LIVE without ENABLE_LIVE_TRADING=true in

How to verify the project is healthy

Run these from the repo root any time you want to confirm nothing is broken:

```
.venv/bin/pytest
```

```
.venv/bin/python -m kdtb.main
```

```
.venv/bin/python scripts/ingest_disclosures.py --date $(date +%Y-%m-%d)
```

What Claude built across the autonomous loops

Loop 1 (M4 + M5):

- **data/market_data_client.py** — pykrx wrapper for OHLCV + event-time returns
- **scripts/run_event_study.py** — batched event study CLI
- **strategy/major_supply_contract.py** — M5 strategy engine (was placeholder)

Loop 2 (cost model + M3 prep + paper broker):

- **backtest/cost_model.py** — Korean equity costs
- **backtest/metrics.py** — per-trade metrics
- **interpretation/llm_prompts.py**, **extraction_validator.py**, **llm_client.py**
- **broker/base.py** + **broker/paper_broker.py**

Loop 3 (deterministic extraction + filtered event study):

- **data/dart_client.py** — new `fetch_document_text()` + ZIP/HTML/UTF-8 extractor
- **interpretation/deterministic_parser.py** — regex parser for **■■■■■■■■■■■■■■■** (voluntary + mandatory variants)
- **storage/extraction_store.py**, **scripts/parse_supply_contracts.py**, **scripts/run_filtered_event_study.py**
- 16 new test cases

Loop 4 (end-to-end signal generation + parser refinement):

- Parser refinement: distinguishes 'value undisclosed' (literal dash) from 'missing entirely'. 0 blocked rows now.
- **storage/signal_store.py + scripts/run_strategy.py**
- 331 candidate signals stored in DB at default threshold (Loop 4 snapshot)
- 4 new test cases

Loop 7 (codified the walk-forward filters into the strategy):

- Added `counterparty_name` + `counterparty_type` fields to Extraction schema, with idempotent ALTER TABLE migration in `init_db`.
- Parser now classifies counterparty (government / large_corp_korean / other_korean_corp / foreign / unknown) using the same regex heuristic the subgroup analysis used.
- Backfilled `counterparty_type` for all 3,531 existing extractions (94 government, 154 foreign, 331 large_corp_korean, 439 other_korean_corp, 2513 unknown).
- Added `kospi_only` and `excluded_counterparty_types` params to `MajorSupplyContractStrategy`. Defaults in config: `kospi_only: true`, `excluded_counterparty_types: ["government"]`.
- **Real PnL improvement** from the filters (still on the same 3,412 OK extractions):

Variant	n	T+1 net	T+5 net	T+5 win%	PF 5d	Notes
v0 — no filters	3,377	-0.12% minus	+0.35%	44.1%	1.10	baseline (Loop 5)
v1 — ratio ≥ 0.08	2,149	-0.07% minus	+0.22%	42.9%	1.06	CLAUDE.md default
v2 — KOSPI + skip-gov	684	+0.17%	+0.67%	46.6%	1.25	Loop-7 filters

The v2 strategy nearly tripled T+5 net mean over v1 (+0.67% vs +0.22%), pushed win rate from 42.9% to 46.6%, and lifted profit factor from 1.06 to 1.25. T+1 mean turned positive for the first time. Median is still

−0.31% at T+5 — most individual trades still lose, but winners are bigger.

Honest caveats: 684 events over 24 months is ~28/month. Win rate is still <50%. PF 1.25 means you win $\blacksquare 1.25$ for every $\blacksquare 1$ lost — thin margin once execution slippage and the 18 blocked rows drift the numbers. This is "real progress", not "ready to trade".

- 92 tests still passing (+4 new for the filters).
- Tests cover: kospi_only rejects KOSDAQ, kospi_only off lets KOSDAQ through, excluded_counterparty_types rejects matching, None counterparty_type still passes.

Loop 8 (walk-forward validation of v0/v1/v2):

The v2 aggregate looked promising; this loop verified that the improvement holds across 4 non-overlapping 6-month windows (the real test of "edge vs curve-fit").

Window	v0 T+5 / PF (n)	v1 T+5 / PF (n)	v2 T+5 / PF (n)
Jul24-Dec24	+0.04% / 1.01 (785)	−0.38% / 0.90 (498)	+1.57% / 1.58 (179) «
Jan25-Jun25	+0.59% / 1.23 (713)	+0.51% / 1.19 (463)	+1.11% / 1.65 (138) «
Jul25-Dec25	+0.41% / 1.15 (930)	+0.67% / 1.25 (573)	−0.55% / 0.79 (200)
Jan26-Jun26	+0.45% / 1.10 (906)	+0.15% / 1.03 (591)	+1.01% / 1.28 (163) «
Aggregate	+0.37% / 1.11 (3,334)	+0.25% / 1.07 (2,125)	+0.72% / 1.27 (680)
Positive windows	4/4	3/4	3/4

Verdict: v2 holds up. 3 of 4 windows positive, average +1.23% in the three winning windows, −0.55% in the losing one. The losers are smaller in magnitude than the winners. Aggregate T+5 net mean +0.72%, profit factor 1.27, win rate 47.1%. v1 is consistently the worst of the three — the ratio filter alone removes signal without adding edge. The KOSPI + skip-government conjunction is what's doing the work.

Still not a strategy ready for real money. 47% win rate, one losing window in four, PF 1.27 ($\blacksquare 1.27$ won per $\blacksquare 1$ lost) — thin margin once execution slippage drifts the numbers. But it IS the first thing in this project that survives walk-forward validation. The next moves (paper broker simulation, then ML on top) are now standing on real ground.

Reproduce with: `.venv/bin/python scripts/walk_forward.py`

Loop 9 (realistic-execution backtest — the painful finding):

Re-priced the 684 v2 signals under three entry/exit assumptions. The "idealized" scenario matches the event study (buy event-day close, sell T+5 close), which assumes a fill we usually *can't* get as a retail trader.

Scenario	Mean	Median	Win%	PF	$\blacksquare$ /trade (on $\blacksquare 30k$)	Total over 684 trades
idealized (t0 close → t+5)	+0.67%−0.31%		46.6%	1.25	+ $\blacksquare 201$	+ $\blacksquare 137,779$
realistic (t+1 close → t+5)	+0.14%−0.82%		42.0%	1.05	+ $\blacksquare 43$	+ $\blacksquare 29,140$
conservative (t+2 close → t+5)	+0.32%−0.31%		44.2%	1.15	+ $\blacksquare 97$	+ $\blacksquare 66,078$

The painful finding: under realistic execution (entering at the T+1 close instead of the event-day close), **most of v2's edge disappears**. Mean net per trade falls from +0.67% to +0.14%, PF from 1.25 to 1.05, win rate from 47% to 42%. Total realized PnL across 684 trades in 24 months would be roughly +\$29,000 — ~\$42/trade. Easily wiped out by execution slippage we haven't modeled.

Distribution shows why: 52% of v2 trades lose money under realistic execution (20% lose more than 5%). The 16% of trades that gain >5% just barely outweigh the losers. This is a strategy whose entire "edge" is concentrated in the event-day reaction. If you can't trade at the close, you don't get the edge.

What this means strategically: the major-supply-contract drift is real but largely captured by the time a retail trader can act. Three realistic paths forward: (a) build infrastructure to react WITHIN seconds of DART publication — that's how Korean HFT desks trade these. CLAUDE.md explicitly excludes this as a non-goal. (b) Test other event types that may have longer-lived reactions: buybacks, dilutive financing, halt/resumption. (c) Accept the verdict and document the negative result as the project's deliverable. The deterministic-pipeline + walk-forward methodology is itself valuable; the strategy choice was the wrong horse.

Reproduce: `.venv/bin/python scripts/run_paper_backtest.py`

Per-trade detail: `data/paper_backtest_v2.csv`

Loop 10 — multi-event-type meta-analysis (the comprehensive answer)

Fetches event-study data for 6 additional event categories besides `supply_contract`, ran the same walk-forward + realistic-execution analysis on each, then adversarially verified every positive finding through three lenses (`sample_size`, `execution_realism`, `regime_stability`). Total: 17 agents in a parallel workflow.

Category	n	T+5 net (idealized)	T+5 net (realistic)	PF realistic	WF +	Verdict
bonus_issue	273	+3.94%	+1.89%	1.54	4/4	positive but fragile (n too small)
buyback	2,093	+1.62%	+0.26%	1.13	4/4	execution kills 84% of edge
supply_contract	3,495	+0.39%	+0.13%	1.04	4/4	execution kills 67% of edge
rights_offering	2,528	+0.25%	+0.18%	1.05	2/4	noise (median −0.94%)
convertible_bond	1,864	+0.85%	+0.04%	1.01	2/4	decaying, monotonic to negative
halt_resumption	1,319	−2.62%	−1.20%	0.72	1/4	consistently negative long
shareholder_change	834	−2.38%	−1.93%	0.52	0/4	consistently negative long

Adversarial verification verdict: ZERO categories pass. Across seven Korean disclosure event categories tested with walk-forward validation and three adversarial verification lenses, zero categories survive strict tradable-edge criteria. `Bonus_issue` posts the highest realistic T+5 net mean at +1.89% with PF 1.54 and 4/4 positive walk-forward windows but fails on sample size (n=273, only 114 unique stocks, one window of n=50 carries the aggregate). `Buyback` (n=2,093) and `supply_contract` (n=3,495) show genuine idealized edge that collapses 84% and 67% respectively under T+1 realistic execution, ending at realistic means of just +0.26% and +0.13%. `Halt_resumption` (−1.20%, PF 0.72, 23% win rate) and `shareholder_change` (−1.93%, PF 0.52, 0/4 positive windows) produce the cleanest negative signals and should be implemented as deterministic long-side blacklists in the risk engine.

Recommended next steps (from the workflow synthesis):

- **Extend bonus_issue dataset 3-5 years** back. Only reconsider paper trading if the worst walk-forward window stays above +0.30% mean with PF > 1.10 after extension. Bonus_issue is the only category with realistic-execution edge above noise; needs ~4× the current n to be statistically defensible.
- **Implement halt_resumption + shareholder_change as long-side blacklists** in the risk engine. These are the strongest negative findings — turning them into a deterministic do-not-trade filter is a tradable risk control even when no profitable signal exists.
- **Formalize the negative result as the Milestone 4 deliverable:** event-study methodology, walk-forward protocol, adversarial verification framework, per-category findings as the project's research output.
- **Defer M5b/M6/M7/M8/M9** (ML filter, broker integration, paper trading, live tiny) until either the extended bonus_issue dataset confirms edge OR a new untested category (earnings surprise proxy, contract cancellation, dilutive financing subtypes) clears the same adversarial bar.

Workflow JSON snapshot: data/multi_event_meta_2026-06-26.json

Reproduce per-category: .venv/bin/python scripts/analyze_event_category.py --category <X>

Cross-category table: .venv/bin/python scripts/summarize_all_categories.py

Loop 11 — halt/shareholder blacklist in the risk engine

Per Loop-10's findings (halt_resumption and shareholder_change are consistently negative-return event categories), implemented a deterministic do-not-trade filter in the risk engine. When the strategy generates a long signal, the risk engine now queries SQLite to check whether the subject stock has had a disclosure in any blacklisted category within the lookback window (default 30 days). If yes, the signal is rejected with reason recent_negative_event:<category>.

Architecture:

- **data/event_categories.py** — shared CATEGORIES dict (extracted from run_event_study.py) so the event-study runner and the risk engine use the SAME patterns. DEFAULT_NEGATIVE_CATEGORIES = [halt_resumption, shareholder_change].
- **risk/event_blacklist.py** — EventBlacklist class wraps a SQLite connection, exposes has_recent_negative_event(stock_code, now, lookback_days) → category_name | None. ~5ms per query on the 514k-row table; no index needed.
- **risk/limits.py** — added blacklist_lookback_days: int = 30 and blacklisted_event_categories: list[str] = [halt_resumption, shareholder_change].
- **risk/checks.py** — new check_negative_event_blacklist() function. Silently skipped if blacklist is None (backwards-compatible with existing tests).
- **risk/risk_engine.py** — RiskEngine now accepts optional blacklist param in its constructor. Default None preserves the existing API.

Validation:

- 17 new test cases covering: detection of both blacklist categories, lookback window boundary, different stock no-match, revision exclusion, supply_contract not in default list, custom category list, unknown category raises, empty stock_code returns None, zero lookback returns None, future event excluded (no look-ahead), picks most-recent when multiple.
- Full suite: 92 → **108 tests passing**. Zero regressions in existing risk tests (the optional blacklist=None default is backwards compatible).
- **Applied to the live 693 v2 stored signals:** 7 (1.0%) would be filtered out — 4 by shareholder_change, 3

by halt_resumption. Rare-fire but high-conviction defensive filter.

Loop 12 — bonus_issue extension result: the methodology was right

Extended the DART dataset back to 2021-12 (added 36 months), then re-ran the bonus_issue event study on the larger sample. Loop-10's adversarial verifier had refuted the bonus_issue claim on sample_size and regime_stability grounds; this loop tested whether more data would confirm or rescue the original finding.

Metric	2yr (n=273)	5yr (n=828)	Change
T+1 net mean	+1.74%	+0.74%	−57%
T+5 net mean	+3.94%	+1.23%	−69%
T+5 median	+1.69%	−0.47%	FLIPPED to NEGATIVE
T+5 win%	57.9%	47.5%	Below 50% now
T+5 profit factor	1.92	1.25	−35%
Realistic T+5 mean	+1.89%	+0.23%	−88%
Realistic PF	1.54	1.06	Essentially breakeven

The verdict: bonus_issue's apparent edge was small-sample artifact, exactly as Loop 10's adversarial verification predicted. Median return FLIPPED from +1.69% to −0.47%, meaning the majority of bonus issues now lose money at T+5. Realistic execution collapses to +0.23% with PF 1.06 — essentially breakeven after costs. This is the same regression-to-mean pattern supply_contract showed going from 90-day to 24-month data.

Methodology validation: the adversarial-verification workflow from Loop 10 correctly flagged bonus_issue as fragile. Two of three lenses (sample_size, regime_stability) refuted the positive verdict on the smaller sample, and the larger dataset confirms. The system is now demonstrating that it can distinguish real signal from noise even when intermediate results look exciting — exactly what an intellectually honest research framework should do.

Where this leaves us: no event category in this project has been shown to have tradable edge after realistic execution costs. The deterministic-pipeline + walk-forward + adversarial-verification methodology IS the project's deliverable, plus a hard-won research finding: *Korean disclosure information is incorporated into prices too efficiently by event-day close for retail entry timing (T+1 close) to capture it.* The halt/shareholder blacklist (Loop 11) remains a genuinely useful defensive risk filter.

DB now: 1,119,270 disclosures across 2021-12-29 to 2026-06-24 (4.5 years). Reproduce: `.venv/bin/python scripts/run_event_study.py --category bonus_issue`

Reproduce: `.venv/bin/python -c "from kdtb.risk import EventBlacklist; ..."`

Loop 5 (24-month backfill + scale-out + the negative finding):

- CLAUDE.md updated: ML/LLM allowed for extraction + feature enrichment + filtering, but NOT in the order-placing decision. New **Milestone 5b** added (ML-enriched signal filter).
- **scripts/backup_db.sh** — gzipped SQL dumps to data/backups/, keeps last 7, ready for cron.
- **24-month DART backfill:** 111,622 → **514,304 disclosures** across 488 trading days (Jun 2024 – Jun 2026).

- **Re-ran deterministic parser:** 533 → **3,412 OK extractions** (96.6% success rate held up at scale, 0 blocked).
- **Re-ran event study + threshold sweep:** the 30%+ edge from Loop 3 (T+5 +3.10%, PF 1.93) collapsed to T+5 +0.50% / PF 1.21 on 8× more data. Median negative at every threshold.
- **Event study script now checkpoints CSV every 100 stocks** — one of yesterday's runs hung overnight; we no longer lose hours of fetches.

Suite: 28 → 43 → 68 → 84 → **88** passing tests. Zero regressions across five loops.

Honest caveats

- The naive event study uses title-only filtering. It does not separate small contracts from large ones, so noise dominates. M3 (LLM extraction) is needed to filter by contract/revenue ratio.
- The event date used is the DART receipt date (YYYYMMDD). Intraday timing of the disclosure is not captured. If a disclosure is filed at 14:00 KST, our 'T+1' return actually includes the price reaction from 14:00 to next-day close, not a clean next-day-only return.
- Returns are raw close-to-close, not benchmark-adjusted. A market-wide rally on event days would inflate the mean.
- pykrx scrapes the KRX website. It's free but can be rate-limited. We sleep 0.15s between calls.
- LIVE_TINY broker code is not yet implemented. Even with KIS credentials, no real orders can be placed yet.